COSMETIC PRODUCT MANAGEMENT SYSTEM

Name	:	M.G. Vishnukumar.
Roll no	:	24EC097.
Register No	:	921724104179.
College	:	Sethu Institute Of Technology.
Department	:	ECE.
Trainer Name	:	Jagadeesh K (Senior technical trainer).

ABSTRACT :

The Cosmetic Product Management System is a simple, efficient console-based application developed in C language for managing cosmetic inventory in small retail shops. The system allows users to Add, View, Search, Update, Delete products, view Low Stock Alerts, and generate basic Inventory Reports. All data is permanently stored in a binary file (cosmetics.dat). This short and clean version contains only essential features, making it easy to understand, maintain, and submit as a mini/final year project. The program uses structures and file handling concepts of C programming.

Keywords: C Programming, Inventory Management, File Handling, Console Application

1.INTRODUCTION :

Managing cosmetic products manually leads to errors, stock mismatches, and expiry losses. This project provides a simple computerized solution using C language. The system helps shopkeepers keep track of products with a clean menu-driven interface and automatic file saving. It is developed as a short, easy-to-understand version suitable for academic submission.

2. OBJECTIVES :

- To develop a simple menu-driven cosmetic inventory system in C.
- To perform Add, View, Search, Update, and Delete operations.
- To provide Low Stock Alert (quantity ≤ 15).
- To generate basic inventory report with total value.
- To ensure data is saved permanently using file handling.
- To keep the code short, clean, and easy to understand.

3.PROBLEM STATEMENT :

Small cosmetic shops face problems like:

- Manual errors in stock records
- Difficulty in finding products quickly
- No alert for low stock items
- Loss of data when register is lost
- Time-consuming calculations for total stock value

This system solves all these issues with a simple and fast C program.

4. SCOPE OF THE PROJECT :

- Manage up to 100 cosmetic products
- Add, View, Search, Update (Name), Delete
- Low stock alert and total value report
- Permanent data storage in file

Out of Scope:

- Graphical interface
- Multi-user login
- Advanced features (discount, sorting, billing)

5. SYSTEM REQUIREMENTS Hardware :

• Any computer with 4 GB RAM and 500 MB free space.

Software:

- ✓ Operating System: Windows / Linux
- ✓ Compiler: GCC / Turbo C / Code::Blocks
- ✓ No extra software or libraries required

6.SYSTEM DESIGN Data Structure Used :

File Used: cosmetics.dat (Binary file)

```c
typedef struct {
    int id;
    char name[50];
    char brand[30];
    char category[30];
    float price;
    int qty;
    char expiry[12];
} Product;
```

- **Maximum Products:** 100

7. METHODOLOGY :

Modular programming approach is used.

- One function for each operation
- File handling (fread & fwrite) for data persistence
- Structure array for storing products
- Simple input validation using fgets and strcspn

8. IMPLEMENTATION :

The complete project is implemented in a single short C file (cosmetic_short.c).

Main Functions:

- add() – Add new product with auto ID
- view() – Display all products in table
- search() – Search by ID or Name
- update() – Update product name
- del() – Delete product
- lowStock() – Show items with qty ≤ 15
- report() – Show total products and total value
- load() & save() – File handling

9.WORKING OF THE PROJECT :

The Cosmetic Product Management System is a menu-driven console application developed in C language. It follows a simple and continuous working flow as explained below:

Startup Process :

- ✓ When the program is executed, the load() function reads all previously saved products from the binary file cosmetics.dat into the structure array p[].
- ✓ The total number of products is stored in variable count.
- ✓ The main menu is displayed immediately.

Main Working Flow :

The program runs in an **infinite while loop** until the user selects option 8 (Exit). At every step:

- **Main Menu** is shown with 8 options.
- User enters choice (1–8).
- The corresponding function is called using switch-case.
- After completing the operation, the program waits for the user to press Enter and returns to the main menu.

Detailed Working of Each Module :

- **Add Product (Option 1)**: Auto-generates next Product ID → Takes input for Name, Brand, Category, Price, Quantity and Expiry Date → Stores in structure array → Immediately saves to file using save().
- **View All Products (Option 2)**: Displays all products in a neat tabular format using a for loop.
- **Search Product (Option 3)**: User chooses search by ID or Name → Program scans the array and shows matching records.
- **Update Product (Option 4)**: User enters Product ID → If found, program allows updating the Product Name (Enter key skips update) → Data is saved back to file.
- **Delete Product (Option 5)**: User enters Product ID → Matching product is removed by shifting remaining records → File is updated.
- **Low Stock Alert (Option 6)**: Scans the entire array and displays only those products whose quantity is ≤ 15.
- **Report (Option 7)**: Calculates total number of products and total inventory value (Price × Quantity) → Shows current date using time() function.
- **Exit (Option 8)**: Calls save() function to permanently store data in cosmetics.dat → Program terminates with "Thank You!" message.

Data Persistence :

All data is stored permanently using binary file handling (fread and fwrite). Even if the program is closed and reopened, all products remain safe.

Program Termination :

The program ends only when the user selects option 8. No data is lost because saving happens automatically after every Add, Update, or Delete operation.

COMPLETE SOURCE CODE :

```
/*****************************************************************************
**************
 *  COSMETIC PRODUCT MANAGEMENT SYSTEM
 *  All essential features + File Save/Load

*****************************************************************************
*************/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX 100
#define FILE_NAME "cosmetics.dat"

typedef struct {
    int id;
    char name[50];
    char brand[30];
    char category[30];
    float price;
    int qty;
    char expiry[12];
} Product;

Product p[MAX];
int count = 0;

void load();
void save();
void add();
void view();
void search();
void update();
void del();
void lowStock();
void report();

int main() {
    load();
    int ch;
    while(1) {
        printf("\n=== COSMETIC MANAGEMENT SYSTEM ===\n");
        printf("1. Add Product\n2. View All\n3. Search\n4. Update\n5. Delete\n6. Low Stock
Alert\n7. Report\n8. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &ch);
```

```
        getchar();

        switch(ch) {
            case 1: add(); break;
            case 2: view(); break;
            case 3: search(); break;
            case 4: update(); break;
            case 5: del(); break;
            case 6: lowStock(); break;
            case 7: report(); break;
            case 8: save(); printf("\nThank You!\n"); exit(0);
            default: printf("Invalid choice!\n");
        }
        printf("\nPress Enter...");
        getchar();
    }
}

void load() {
    FILE *f = fopen(FILE_NAME, "rb");
    if(f) {
        count = fread(p, sizeof(Product), MAX, f);
        fclose(f);
    }
}

void save() {
    FILE *f = fopen(FILE_NAME, "wb");
    if(f) {
        fwrite(p, sizeof(Product), count, f);
        fclose(f);
    }
}

void add() {
    if(count >= MAX) { printf("Database Full!\n"); return; }
    Product t;
    t.id = (count==0) ? 1001 : p[count-1].id + 1;

    printf("ID: %d\n", t.id);
    printf("Name     : "); fgets(t.name,50,stdin); t.name[strcspn(t.name,"\n")]=0;
    printf("Brand    : "); fgets(t.brand,30,stdin); t.brand[strcspn(t.brand,"\n")]=0;
    printf("Category : "); fgets(t.category,30,stdin); t.category[strcspn(t.category,"\n")]=0;
    printf("Price    : "); scanf("%f",&t.price);
    printf("Quantity : "); scanf("%d",&t.qty);
    printf("Expiry (YYYY-MM-DD): "); scanf("%s",t.expiry);
    getchar();

    p[count++] = t;
    save();
```

```
    printf("Product Added Successfully!\n");
}

void view() {
    if(count==0) { printf("No products!\n"); return; }
    printf("\nID   Name                 Brand       Category     Price    Qty   Expiry\n");
    printf("-----------------------------------------------------------------------------\n");
    for(int i=0; i<count; i++)
        printf("%-4d %-20s %-12s %-12s ₹%-7.2f %-5d %s\n",
              p[i].id, p[i].name, p[i].brand, p[i].category, p[i].price, p[i].qty, p[i].expiry);
}

void search() {
    int id, found=0;
    char name[50];
    printf("Search by 1.ID  2.Name : ");
    int ch; scanf("%d",&ch); getchar();

    if(ch==1) {
        printf("Enter ID: "); scanf("%d",&id);
        for(int i=0;i<count;i++) if(p[i].id==id) {
            printf("Found: %s | %s | ₹%.2f | Qty:%d\n", p[i].name, p[i].brand, p[i].price, p[i].qty);
            found=1;
        }
    } else {
        printf("Enter Name: "); fgets(name,50,stdin); name[strcspn(name,"\n")]=0;
        for(int i=0;i<count;i++) if(strstr(p[i].name, name)) {
            printf("Found: %s | %s | ₹%.2f | Qty:%d\n", p[i].name, p[i].brand, p[i].price, p[i].qty);
            found=1;
        }
    }
    if(!found) printf("Not found!\n");
}

void update() {
    int id, i;
    printf("Enter Product ID: "); scanf("%d",&id);
    for(i=0;i<count;i++) if(p[i].id==id) break;
    if(i==count) { printf("Not found!\n"); return; }

    printf("Update Name (Enter=skip): "); char temp[50]; fgets(temp,50,stdin);
if(temp[0]!='\n') strcpy(p[i].name, temp);
    // You can add more fields if needed
    printf("Updated!\n");
    save();
}

void del() {
    int id, i;
    printf("Enter ID to delete: "); scanf("%d",&id);
```

```
    for(i=0;i<count;i++) if(p[i].id==id) break;
    if(i==count) { printf("Not found!\n"); return; }
    for(int j=i; j<count-1; j++) p[j]=p[j+1];
    count--;
    save();
    printf("Deleted!\n");
}

void lowStock() {
    printf("\nLOW STOCK ITEMS:\n");
    int found=0;
    for(int i=0;i<count;i++) if(p[i].qty <= 15) {
        printf("ID:%d | %s | Qty:%d\n", p[i].id, p[i].name, p[i].qty);
        found=1;
    }
    if(!found) printf("No low stock items.\n");
}

void report() {
    float total = 0;
    for(int i=0;i<count;i++) total += p[i].price * p[i].qty;
    printf("\n=== REPORT ===\n");
    printf("Total Products : %d\n", count);
    printf("Total Value    : ₹%.2f\n", total);
    char date[12];
    time_t t = time(NULL);
    strftime(date,12,"%Y-%m-%d",localtime(&t));
    printf("Generated on   : %s\n", date);
}
```

OUTPUT SCREEN :

```
Output                                                                    Clear

=== COSMETIC MANAGEMENT SYSTEM ===
1. Add Product
2. View All
3. Search
4. Update
5. Delete
6. Low Stock Alert
7. Report
8. Exit
Enter choice: 1
ID: 1001
Name     : Lakme Absolute Lipstick
Brand    : Lakme
Category : Lipstick
Price    : 599.50
Quantity : 45
Expiry (YYYY-MM-DD): 2027-08-15
Product Added Successfully!

Press Enter...

=== COSMETIC MANAGEMENT SYSTEM ===
1. Add Product
2. View All
3. Search
4. Update
5. Delete
6. Low Stock Alert
7. Report
8. Exit
Enter choice: 2

ID   Name                 Brand       Category     Price    Qty   Expiry
-----------------------------------------------------------------------------
1001 Lakme Absolute Lipstick Lakme       Lipstick     ₹599.50  45    2027-08-15

Press Enter...
```

Output Clear

```
=== COSMETIC MANAGEMENT SYSTEM ===
1. Add Product
2. View All
3. Search
4. Update
5. Delete
6. Low Stock Alert
7. Report
8. Exit
Enter choice: 1
ID: 1002
Name     : Maybelline Fit Me Foundation
Brand    : Maybelline
Category : Foundation
Price    : 899.00
Quantity : 8
Expiry (YYYY-MM-DD): 2026-06-30
Product Added Successfully!

Press Enter...

=== COSMETIC MANAGEMENT SYSTEM ===
1. Add Product
2. View All
3. Search
4. Update
5. Delete
6. Low Stock Alert
7. Report
8. Exit
Enter choice: 2

ID   Name                  Brand       Category     Price    Qty   Expiry
-----------------------------------------------------------------------------
1001 Lakme Absolute Lipstick Lakme        Lipstick     ₹599.50  45    2027-08-15
1002 Maybelline Fit Me Foundation Maybelline   Foundation   ₹899.00  8     2026-06-30
```

Output

Clear

```
Press Enter...

=== COSMETIC MANAGEMENT SYSTEM ===
1. Add Product
2. View All
3. Search
4. Update
5. Delete
6. Low Stock Alert
7. Report
8. Exit
Enter choice: 3
Search by 1.ID  2.Name : 2
Enter Name: Lakme
Found: Lakme Absolute Lipstick | Lakme | ₹599.50 | Qty:45

Press Enter...

=== COSMETIC MANAGEMENT SYSTEM ===
1. Add Product
2. View All
3. Search
4. Update
5. Delete
6. Low Stock Alert
7. Report
8. Exit
Enter choice: 4
Enter Product ID: 1001
Update Name (Enter=skip): Updated!

Press Enter...
```

Output

Clear

```
Press Enter...

=== COSMETIC MANAGEMENT SYSTEM ===
1. Add Product
2. View All
3. Search
4. Update
5. Delete
6. Low Stock Alert
7. Report
8. Exit
Enter choice: 5
Enter ID to delete: 1002
Deleted!

Press Enter...
=== COSMETIC MANAGEMENT SYSTEM ===
1. Add Product
2. View All
3. Search
4. Update
5. Delete
6. Low Stock Alert
7. Report
8. Exit
Enter choice: 6

LOW STOCK ITEMS:
No low stock items.

Press Enter...

=== COSMETIC MANAGEMENT SYSTEM ===
1. Add Product
2. View All
3. Search
4. Update
5. Delete
```

Output

Clear

```
LOW STOCK ITEMS:
No low stock items.

Press Enter...

=== COSMETIC MANAGEMENT SYSTEM ===
1. Add Product
2. View All
3. Search
4. Update
5. Delete
6. Low Stock Alert
7. Report
8. Exit
Enter choice: 7

=== REPORT ===
Total Products : 1
Total Value    : ₹26977.50
Generated on   : 2026-02-25

Press Enter...

=== COSMETIC MANAGEMENT SYSTEM ===
1. Add Product
2. View All
3. Search
4. Update
5. Delete
6. Low Stock Alert
7. Report
8. Exit
Enter choice: 8

Thank You!
```

10. ADVANTAGES :

- Very short and easy to understand
- No extra libraries needed
- Data is automatically saved
- Fast execution
- Perfect for academic project submission
- Low memory usage
- Easy to modify and extend

11. LIMITATIONS :

- Can update only Name (not price/qty)
- No sorting feature
- Low stock level is fixed (≤15)
- Console-based only (no GUI)
- Single user only

12. FUTURE ENHANCEMENTS :

- Full update for all fields
- Add sorting by price/expiry
- Sales billing module
- Graphical User Interface
- User login system
- Export report to PDF

13. CONCLUSION :

The Cosmetic Product Management System (Short Version) has been successfully developed and tested. It fulfills all basic requirements of inventory management with clean and short code. This project helped in learning structures, file handling, and modular programming in C. It is ready to use in small cosmetic shops and is ideal for project submission.

14. REFERENCES :

1. Yashavant Kanetkar, “Let Us C”, BPB Publications.
2. Brian W. Kernighan & Dennis M. Ritchie, “The C Programming Language”.
3. GeeksforGeeks – File Handling in C.
4. Tutorialspoint – C Structures and Inventory System Examples.